Quarb Implementation Guide

The Quarb Project

Version: Draft 0.1 (walking skeleton)

Contents

1 About This Guide **2**

2 Architecture **3**
2.1 Workspace Layout 3
2.2 The Execution Ladder 22
2.3 Current Query Subset 24

3 The Adapter Surface **30**
3.1 The Live Trait 30
3.2 Worked Example: the Filesystem Adapter 33
3.3 The Planned Surface 33

4 Building and Running **35**

5 Planned Integration Paths **36**

Chapter 1

About This Guide

This guide describes the **Quarb engine**: the Rust implementation of the Quarb query language, its `qua` command-line tool, and the adapter architecture that lets one engine query many data sources. The language itself — its data model, execution model, and syntax — is defined in the *Quarb Language Specification* (the `lang` sibling project); this guide covers only the implementation.

The guide tracks the code. The engine is being built as a series of thin, working slices rather than from a finished design. This document tracks that process: constructs the engine *already implements* are marked **[live]** and documented against the real code; constructs that are specified but *not yet built* are marked **[planned]** and sketch the intended surface. As each slice lands, its **[planned]** entries become **[live]**. The current milestone is a *walking skeleton*: rooted tree navigation over a filesystem.

Central concern: the adapter surface. The one design question this guide most needs to get right is the *adapter surface* — the trait a data source implements to be queried. That surface is deliberately being grown against a concrete first adapter (the filesystem) so that its shape is driven by real navigation problems rather than speculation. Chapter 3 is its home.

Chapter 2

Architecture

The engine follows a conventional compiler-style pipeline. A query string is lexed and parsed into a small AST, which the executor evaluates by driving an *adapter* over the target data:

```
[query string]
      |  lexer          quarb::lexer
      v
[tokens]
      |  parser         quarb::parser
      v
[query AST]             quarb::ast
      |  executor       quarb::exec
      v
[node set]  <-- drives --  [AstAdapter]  --> [data source]
```

The executor never touches the data source directly. It knows only the `AstAdapter` trait (Chapter 3), so the same query runs unchanged over any adapter.

2.1 Workspace Layout

The engine is a Cargo workspace mirroring the intended separation of concerns — the core engine, one crate per adapter, and the CLI:

Crate	Responsibility
`quarb`	The core engine: `lexer`, `parser`, `ast`, `exec`, and the `adapter` module defining `AstAdapter`. No I/O.
`quarb-fs`	The filesystem adapter. Depends on `quarb`.

`quarb-imap`	Remote IMAP mailboxes, queried in place: the maildir mapping over the wire (folders, messages by UID, headers as properties, `::::epoch`), transported by `curl` over subprocess (it speaks IMAP natively) — zero new dependencies. Lazy per message: one LIST, one UID SEARCH per folder, two fetches per touched message. The thread fabric is deliberately absent (indexing it means fetching every header — that is a sync; use mbsync + `quarb-maildir`). Target: `imap[s]://HOST[:PORT][/FOLDER]`; credentials via `QUARB_IMAP_USER`/`PASS` or `~/.netrc`. Read-only. Tested against a bundled deterministic mini IMAP server.
`quarb-json`	The JSON adapter (keys and indices as names, value types as traits). Depends on `quarb` and `serde_json`.
`quarb-gsheet`	Google Sheets on the `quarb-xlsx` mapping (sheets as tables, first-row headers, typed values via `UNFORMATTED_VALUE`), fetched from the Sheets API. Auth: the API rejects gcloud's `cloud-platform` tokens, so the friction-free path is an API key (`?key=` / `QUARB_GSHEET_KEY`; enough for link-readable sheets), with a properly-scoped bearer for private ones. Target: `gsheet://SPREADSHEET_ID`.
`quarb-html`	The HTML adapter (tags as names, block/inline as traits, attributes as properties, anchor fragments resolve). Depends on `quarb` and `scraper`.
`quarb-pdf`	PDF internals as the object graph: indirect objects are nodes named by their declared `/Type`, scalar dictionary entries are properties, references are edges labeled by their key (`->Kids`, `<-Parent`; array references hoist to the array's owner), with the reverse index built at load. Depends on `quarb` and `lopdf`.

`quarb-text`	The shared text-level model — the relational pattern for documents: the per-format crates lower their format into one block-event vocabulary (the atrep shape: kind, taxis, lemma, body, hypograph), and this crate alone implements the adapter. Sections with explicit title (`::lemma`) + body are derived from the flat heading stream by the outline rule; paragraphs, blockquotes (attribution as `::hypograph`), lists, and verbatim blocks (`::::lang`) complete the vocabulary; tables denormalize into nested lists carrying the `<table>` trait. Bare `::` is the flattened prose of the subtree. The apparatus is first class (litogramma's model): `footnote`/`endnote` callouts and bodies share their family name, resolve by (family, onym) into `->footnote`/`->endnote` edges, and carry `<deixis>` (the callout), `<note>` (the body), and `<dangling>` traits; the `aside` family (anchored margin content) rides the same machinery without `<note>`, and margin-spelled footnotes carry `::::form`; `index-mark` nodes expose declared index marks (`::term`) invisible to the prose. Plain text (`.txt`, blank-line paragraphs) mounts here by default; `text:` forces the reading for html/md. Depends on `quarb`.
`quarb-text-docx`	The Word text-level adapter, from declared structure only: outline levels resolved through the style chain (never style names), the accepted view of tracked changes, `w:tblHeader` header rows, the closed cited quote-style set, both note parts (`word/footnotes.xml`, `word/endnotes.xml`), and `XE` field instructions as index marks. Depends on `quarb-text`, `zip`, and `quick-xml`.
`quarb-text-epub`	The EPUB text-level adapter: container to rootfile to spine, chapters through the HTML adapter in spine order (`linear="no"` excluded); the EPUB note vocabulary arrives via `quarb-text-html`. Depends on `quarb-text`, `quarb-text-html`, and `zip`.

`quarb-text-html`	The HTML text-level adapter: soup dropped (nav, chrome, scripts, forms, media), the head read for the page's declared identity (`title`, `lang`, canonical, every `meta`, `rel=tag` links, a JSON-LD block — verbatim as `::::` metadata on the document, and curated into tags-as-traits, `::category`, `::description`, `::author`, `::published`; every node answers `::href` / `::anchor` / `::link`, and `| link` rides them), wrappers transparent, headings flat into the shared derivation, trailing `cite`/`footer`/`figcaption` as quote attribution; notes only from the declared vocabularies (`epub:type` noteref/footnote/endnote/rearnote/marginalia, DPUB-ARIA `role` doc-*), endnote words landing in the endnote family, marginalia in the aside family. Depends on `quarb-text` and `scraper`.
`quarb-text-koine`	The koine adapter — the hub's first rung: a koine/litogramma document (parsed by the `atrep` crate, settled by `kanonizo::tasso`) lowers into the shared block vocabulary through its dialektos-resolved sim names — enclosing headings by nesting depth, both note families and asides by name, index endos as marks, stichoi tables, refs keeping their key as written. Foreign formats arrive through atrep's endomorphosis importers (`atrep::endo`) into their mirror dialects — Markdown, HTML, reStructuredText, Org, djot — read by the same name-keyed mapping (flat `heading-N` endos, flat item runs wrapped, rst footnotes into the shared apparatus). Serves `koine:` (and `text:` on `.atd`/`.atk`, where the routes converge); `.xml` dispatches by declared identity (root namespace, DOCTYPE public id, unambiguous root element) into the TEI/DocBook/JATS/USX/OSIS importers, `?format=` forcing any format; a bare `<article>` and everything nothing imports yet — LaTeX included — refuse naming the working spelling. Depends on `quarb-text` and `atrep`.

`quarb-text-latex`	The LaTeX text-level adapter: the declared structural subset of a `.tex` source — declared sectioning commands, footnotes (the at-callout convention normalized to document-end bodies), the endnotes package on its own counter, Tufte sidenotes into the footnote family (`::::form`), `\marginpar` / `\marginnote` as asides, `\index` marks, quote/quotation, itemize/enumerate, verbatim; prose by drop list and unwrap-by-default, definition argument trains consumed whole. Depends on `quarb-text`.
`quarb-text-markdown`	The Markdown text-level adapter, mapping pulldown-cmark events directly (no HTML round-trip): heading levels, list ordinals, fence languages, and pipe tables come from the source. Depends on `quarb-text` and `pulldown-cmark`.
`quarb-text-pdf`	The PDF text-level adapter — the reader's model from the two declared rungs: the `/Outlines` tree (titles as `::lemma`, destinations bounding each section's extent) and exact-baseline line geometry (bit-for-bit equal `y`, `::::page`/`::::y` metadata; ToUnicode CMaps decoded, one stated kern constant for word gaps). No `paragraph` is ever minted. Depends on `quarb-text`, `quarb-pdf`, and `lopdf`.
`quarb-xml`	The XML adapter (qualified tags as names, attributes as properties, `id`/`xml:id` references resolve; strict well-formedness, no traits). Depends on `quarb` and `quick-xml`.
`quarb-relational`	The shared relational model: tables as root children, rows named by primary key, columns as properties, declared foreign keys driving `~>` resolution (chained hops instead of JOINs), `->`/`<-` crosslinks labeled by column, `<~` reverse resolution, and the table-naming hint for undeclared references. Loading is catalog- eager, rows-lazy: a table materializes on first touch through the driver-supplied fetcher, once, and untouched tables are never read (`::::loaded` reports the state). Engine adapters are thin catalog drivers over it. Depends on `quarb`.

`quarb-sqlite`	The SQLite catalog driver: PRAGMA introspection, storage-class type mapping. Depends on `quarb-relational` and `rusqlite`.
`quarb-objstore`	GCS and S3 buckets as lazy directory trees (`gs://` / `s3://`, delimiter listings, paginated; object content fetched on first read). Composed by default in `qua`: a bucket of JSON / CSV / source files grafts, so `/data/store.json/books/*` crosses from cloud storage into the document mid-path — grafting is this adapter's point. GCS: anonymous for public buckets, gcloud-token auth otherwise. S3: anonymous only in v1 (SigV4 signing is a recorded extension — private buckets refuse).
`quarb-postgres`	The PostgreSQL catalog driver: `information_schema` introspection over `tokio-postgres` (internal current-thread runtime, synchronous surface); native bool/int/float/text, everything else `::text`-cast into the numeric-reading regime. Depends on `quarb-relational`.
`quarb-mysql`	The MySQL/MariaDB catalog driver: `information_schema` introspection over `sqlx` (internal current-thread runtime, synchronous surface; `key_column_usage` carries the foreign keys directly); integer/float/character families native, everything else `CAST(... AS CHAR)` into the numeric-reading regime. Depends on `quarb-relational`.
`quarb-mssql`	The SQL Server catalog driver: `INFORMATION_SCHEMA` plus `sys.foreign_key_columns` (parallel column ids keep composite keys paired), over `tiberius` (pure-Rust TDS, internal current-thread runtime); `bit`/integer/float/character families native, everything else `CAST(... AS NVARCHAR(MAX))`. Target: `mssql://USER:PASS@HOST[:PORT]/DATABASE`. Depends on `quarb-relational`.

`quarb-oracle`	The Oracle catalog driver: the `USER_*` dictionary views over the `oracle` crate (ODPI-C, synchronous); `NUMBER(p,0)` integer, `BINARY_FLOAT`/`DOUBLE` float, character family text, `DATE`/`TIMESTAMP` through an ISO-8601 `TO_CHAR` into the temporal regime, everything else `TO_CHAR`. Target: `oracle://USER:PASS@HOST[:PORT]/SERVICE`. Depends on `quarb-relational`.
`quarb-ldap`	The LDAP directory driver: the DIT *is* the arbor — the base DN is the root, an entry's children are the entries one level below (RDN names), its attributes are properties (multi-valued as lists), `objectClass` values are traits. DN-valued attributes (`member`, `manager`, ...) are native references — `~>` follows, `->` enumerates, `<-member` finds the groups by a filtered search. Over the `ldap3` sync client, anonymous or simple bind. Target: `ldap[s]://[USER:PASS@]HOST[:PORT]/BASE_DN`.
`quarb-cbor`	The CBOR (RFC 8949) adapter, the JSON mapping extended: non-text map keys name nodes by canonical rendering, byte strings are hex scalars (`::::length` is the raw byte count), the standard time tags (0, 1) decode to instants, other tags are transparent with the number at `::::tag`. Traits carry the CBOR type. Binary, by the `.cbor` extension. Depends on `ciborium`.
`quarb-csv`	The CSV/TSV adapter (one `row` node per record, columns as properties, empty cells as missing values that numeric aggregates skip; `::::columns` / `::::n-rows` metadata). Depends on `quarb` and `csv`.
`quarb-xpath`	The XPath 1.0 bridge, both directions: `translate` imports an XPath expression to Quarb, and `export` walks a Quarb query's reflection arbor (the locked vocabulary as the tooling surface) back to XPath — explicit refusals and divergence notes both ways. Binaries `xpath2quarb` / `quarb2xpath`; the export side is differentially tested against `xmllint`.

`quarb-jq`	The jq bridge, both directions: `translate` imports a jq filter (value-projecting; a node mode pairs with `jq 'path(...)'`), and `export` walks the reflection arbor back to a jq filter. Binaries `jq2quarb` / `quarb2jq`; the export side is differentially tested against the `jq` binary.
`quarb-bigquery`	The Google BigQuery driver: the REST API directly (`jobs.query` over a synchronous HTTP client — no cloud SDK, no async runtime), bearer token from `QUARB_BQ_TOKEN` or `gcloud auth print-access-token`, catalog from `INFORMATION_SCHEMA` including the *unenforced* key constraints (declare them and the reference machinery works). The execution ladder is a billing instrument here: BigQuery meters bytes scanned, so lazy loading, pushdown, and `--save` directly reduce cost. Target syntax: `bigquery://PROJECT/DATASET[?account=EMAIL]`.
`quarb-duckdb`	DuckDB, the fifth catalog driver — and the columnar side door: a `CREATE VIEW ... AS SELECT * FROM read_parquet(...)` makes Parquet first-class. Keys from `duckdb_constraints()` (the `information_schema` FK view reports the referencing side only), rows lazy, `raw_query` for the ladder, read-only connection. Bundled build; `.duckdb` / `.ddb` by extension.
`quarb-xlsx`	Spreadsheets (`.xlsx` / `.xls` / `.ods`, via calamine) on the relational model: sheets as tables, first-row headers as columns (positional `cN` for blank/duplicate), rows named by sheet row number, numeric cells typed. No keys — sheets declare none — so resolution needs hints. Dispatched by extension *before* the archive check (workbooks are zips; the sheets are the point).

`quarb-firebase`	The Firebase Realtime Database driver: the RTDB *is* a JSON tree, so the mapping is `quarb-json`'s, fetched lazily one node per shallow GET and cached. Properties are direct fetches (`::score` is one tiny request); `~>` resolves by hint naming a root-relative container (`::parent~>item`). Containers that refuse enumeration (permission-scoped or unbounded — the public Hacker News database's `/v0/item` holds tens of millions of keys) are *opaque*: reachable by name, empty to wildcards — served by the engine's new `children_named` fast path. References are *declared client-side*: a refs document (`{"refs": {"parent": "item", "kids/*": "item"}}`, supplied via `qua --refs`) is the portable mini-catalog a schemaless substrate cannot hold — with it, bare `~>` resolves and `->` crosslinks enumerate (array fields one probed edge per element); an inline hint overrides. Reverse resolution stays empty (it would scan what opaque containers refuse; server-side `.indexOn` queries are the recorded v2 path). Target: `firebase://HOST/PATH[?auth=...]`. Read-only.
`quarb-tree-sitter`	Source code as its literal syntax tree — the syntax level (né `quarb-code`): node kinds as edge names (`//function_item`), tree-sitter *fields* as properties (`::name` on a function is its name), source text as the value, `::::start-line` / `::::kind` / `::::field` metadata; named nodes only. Rust / Python / JavaScript / C by extension, compile-time grammar set. Grafts under composition — `--graft` queries every function in a directory tree at once.

`quarb-code`	Source code at the code level: declaration identifiers are node names (`/lexer/lex`), anonymous constructs keep a normalized cross-language vocabulary (`if`, `switch`, `for`, `call`), everything else dissolves (children hoist). Curated traits (`<function>`, `<type>`, `<loop>`); `::signature` / `::doc` / `::callee`; raw backend kind only at `::::kind` (aliased, closed surface); `->definition` file-local crosslinks. Opt-in via the `code:` target prefix; `code:DIR` grafts every supported source leaf — filesystem into declarations, one namespace.
`quarb-compose`	Adapter *composition*: wrap any adapter and a leaf whose name or content parses (json / xml / html / csv, else a sniff) grafts its parsed arbor as children — `/data/store.json/books/*` crosses from filesystem into JSON mid-path. Marked-up members graft at the DOM by default or, under `DocumentGraft::Text` (`text:` on an archive), as the text level; the source-level grafts (tree-sitter, the code model) sit behind the default `source` feature so the wasm build can leave them out. Grafts are lazy (read + parse on first entry), the inner root is identified with the outer leaf (no phantom node), and locators mark the boundary jar-style (`store.json!/books/1`). `qua`: default for archives, `--graft` for directories.
`quarb-archive`	Zip and tar (`.tar[.gz]`) archives as path trees: entry content as the node value, `::::size` / `::::compressed`; zip contents lazy per entry, tar in one pass. With composition, `.docx` / `.epub` / `.jar` internals are directly queryable.

`quarb-web`

The *web level*: a site as an arbor — `/sites/<host>/pages/<path>` for every page (directories from the URL paths, `<dir>` / `<page>` / `<redirect>`), the declared categories and tags as rows of their own containers, every hyperlink of the DOM resolved into `->link` / `<-link` (`->category`, `->tag`, `<-page`, `<-category`), the degrees, `::pagerank`, `::redlinks`, and `<orphan>` as columns; each page grafts at the text level on entry, and a prose `ref` that leaves its page resolves to the page it names. Rows answer to their kind (`//page`), their segment, its stem, their title. A `WebStore` contract behind it; stores: in-memory from a tarball (`archive`, the wasm `Site`) or a directory (`fs`), SQLite (`sqlite`), PostgreSQL (`postgres`; COPY export, deferred foreign keys), one schema in two dialects. A planner over the reflected query answers on three rungs — Full (a bare count, every conjunct exact), Prefilter (a superset of the enumerating step's matches; the engine re-runs the query over the level scoped to the candidates), Scan — with the text prefilter resting on `page_text.plain` = the graft's own `plain_text()`. The four adapter hooks (`document_by_ref`, `reverse_resolve`, `descendants_named`, `prefetch_links`) let a store answer where the engine would walk. `quarb-web-ingest` (`cli`): `crawl` a directory or archive into a store, `wikipedia` the Wikimedia Enterprise HTML dumps (streamed; links by URL then through redirects; `--tree=path|category`), `analyze`, `index` (pg_trgm). `qua`: `web:<dir|archive|site.db|postgres://…>`, `--explain` / `--no-pushdown`.

`quarb-trace`	The `quarb/trace` viz contract as Rust: the payload types (arbor snapshot, steps, threads, snapshots, window and stubs, positions, ribbons, lookahead verdicts, results) and their kaiv writer; no dependencies, no reader. `quarb::trace::trace` fills one by *replaying* the query rather than instrumenting the evaluator: one hop per live thread through the evaluator's own `navigate_paths` (candidates with the predicates stripped, survivors with them — the fork, the verdicts, the walked edges), `apply_stage` per thread for filters and over the whole context for aggregates, a hand-iterated walk for quantified groups (one frame per depth), then checks the final context against a plain evaluation. A staged suffix marks the steps beyond the committed query (a filter preview); a window selects the slice of a large result list to embed and what the stubs on either side measure. `Doc::trace` in `quarb-session` is the entry; the renderers read the kaiv document (quarbopsis: `quao trace`).
`quarb-maildir`	Maildir (`cur/`+`new/`) and mbox mailboxes: headers as properties (lowercased), the body as the value, `::::epoch` parsing the Date header for range predicates. The thread graph is reference fabric: `::in-reply-to~>` walks to the parent message, `<-in-reply-to` finds the replies, via a Message-ID index. Never touches flags; read-only.
`quarb-firestore`	Google Firestore (native mode): collections and documents alternate as the arbor's levels; map and array fields descend as children, subcollections beside them. `referenceValue` fields are *native* references — the first remote adapter where `~>` and `->` need neither a schema nor a refs file. Lazy per level, REST direct, token via `QUARB_GCP_TOKEN` / gcloud. Target: `firestore://PROJECT/DATABASE[?account=...]`.

`quarb-datastore`	Google Cloud Datastore (Firestore in Datastore mode): kinds and entities, the most relational of the document stores; `keyValue` properties are native references (`~>`/`->` schema-free). Kind list via a `__kind__` query, entities via cursor-paginated `runQuery`, lazy per kind. Target: `datastore://PROJECT[/DATABASE][?account=...]`. Both drivers send the `x-goog-request-params` routing header named databases require.
`quarb-mongodb`	MongoDB: collections hold BSON documents, named by `_id` (ObjectIds as hex); embedded documents and arrays descend as children. DBRef fields (`{$ref, $id}`) are native references (`~>`/`->` schema-free). BSON dates arrive as instants; ObjectId and Decimal128 as strings. Lazy per collection (one `_id`-sorted `find`), reference targets by single `findOne`s; official Rust driver, sync API. Read-only. Target: a standard connection string with the database as the path, `mongodb://HOST[:PORT]/DATABASE[?options]` (or `mongodb+srv://`); a missing `serverSelectionTimeoutMS` defaults to 5 s.
`quarb-kubernetes`	The Kubernetes API, which is already a tree: the root holds one listing per discovered resource (cluster-scoped kinds plus the all-namespaces view, kubectl's `-A`); each namespace object carries its scoped listings beside its own fields; objects descend as their JSON, RFC 3339 strings as instants. `ownerReferences` are native references (`::owner~>` climbs Pod → ReplicaSet → Deployment schema-free; `->node`, `::namespace~>`); unknown property names fall through to `metadata.labels`. Lazy per listing, discovery at connect. Transport is `kubectl get --raw` over subprocess (git-adapter posture, zero new dependencies; `QUARB_KUBECTL` overrides the binary), so contexts and auth behave exactly as kubectl does. Read-only. Target: `k8s:` / `k8s:CONTEXT` (alias `kubernetes:`).

`quarb-gcplogs`	Google Cloud Logging as a bounded snapshot — never a tail: the target must carry `since=` (a gcloud freshness duration or an ISO instant) and/or `limit=`, and an unbounded mount is refused. Entries oldest-first at the root as `/entry` children (the `/row` convention); severity as text property, lowercased trait (`/entry<error>`), and ordered numeric rank (`::::severity`, GCP's enum values); timestamps as typed instants; `::trace` the bare trace id — the `<=>` join key across services. Unknown property names fall through payload top level → labels → resource labels → httpRequest. `filter=` passes a raw Cloud Logging filter to the server (the mount-side cut); `until=` joins it as a timestamp clause. Transport is `gcloud logging read --format=json` over subprocess (kubectl posture; `QUARB_GCLOUD` overrides — the bottled test fixture's hook; `account=` picks the gcloud account). Read-only. Target: `gcl:PROJECT?since=1h&filter=…&limit=N` (alias `gcplogs:`).
`quarb-cwlogs`	AWS CloudWatch Logs, the gcplogs design sheet over SigV4 (shared `quarb-aws` signing, credential chain, region resolution): bounded snapshots (`since=`/`limit=` mandatory), log groups as short-named root children with lazily fetched `/entry` events (or one group's entries at the root when the target names it), JSON messages decoded with property fallthrough (`::level`, `::requestId` — the cross-group join key), `filter=` as a server-side CloudWatch filter pattern, `::::group`/`::::stream`/`::::received`/`::::id` metadata. `endpoint=` overrides the URL (LocalStack; the bottled `cwlogs-server.py` fixture). Read-only. Target: `cwl:[GROUP]?since=1h&filter=…&limit=N` (alias `cloudwatch:`).

`quarb-azlogs`	Azure Monitor Logs through the Log Analytics KQL endpoint — the adapter composes the KQL itself (bounded table scan, `filter=` as a `| where` clause, `take` from `limit=`); `table=` and a bound are mandatory. One table's rows at the root as `/entry`, a comma list as root children per table. Columns are properties (`datetime` as instants, `dynamic` decoded into subtrees with fallthrough); `::OperationId` is the correlation key; the default value is `Message` or the `Name`+ `ResultCode` request line. Bearer token from `$AZURE_LOG_TOKEN` or the az CLI (`QUARB_AZ`); `endpoint=` overrides (the bottled `azlogs-server.py`). Read-only. Target: `azl:WORKSPACE?table=…&since=1h&limit=N` (alias `azlogs:`).
`quarb-cflogs`	Cloudflare edge logs, Logpull-shaped: `GET zones/{zone}/logs/received` NDJSON, one flat record per request, fields as properties under their own names plus minted `::ray` and `::timestamp`; the default value is the request line; `::::cache` the cache status. `since=` is mandatory (Logpull demands a range; `until` clamps a minute behind now). The Ray ID is what the origin sees as `cf-ray`, so mounting the edge next to origin logs makes request reconstruction cross-provider — one `<=>` on the ray, another on the origin's own trace. `$CLOUDFLARE_API_TOKEN`; `endpoint=` replays Logpush archives (the bottled `cflogs-server.py`). Read-only. Target: `cfl:ZONE_ID?since=1h&limit=N` (alias `cflogs:`).

`quarb-ddlogs`	Datadog Logs through the v2 search API (`POST /api/v2/logs/events/search`, cursor-paginated): bounded snapshots (`since=`/`limit=` mandatory), `query=` as Datadog's own search syntax run server-side. Severity is Datadog's `::status` plus the family trait and ordered `::::severity` rank; `::trace` is minted from `dd.trace_id`; tags split into fallthrough properties (`env:prod` answers `::env`); custom attributes navigate as children. Auth `$DD_API_KEY`/`$DD_APP_KEY`, site from `site=`/`$DD_SITE`; `endpoint=` overrides (the bottled `ddlogs-server.py`). Read-only. Target: `ddl:?since=1h&query=…&limit=N` (alias `datadog:`).
`quarb-github`	GitHub, three shapes in their native registers. The tree mirrors github.com URLs: users/orgs at the root (addressed by login — the root does not enumerate), repositories as their children, `files` / `issues` / `pulls` / `releases` sections below; file content and issue bodies are node values. The social graph is edge fabric, API-backed both ways: `follows`, `starred`, `owner`, `parent` (`<-parent` = the forks), `author`/`assignee`, `org`, and `contributor` — the first adapter with true `backlinks`, and its contributor edge carries data (`$-::contributions`). Counts and scalars are properties (`::stars`, timestamps as instants); flags, issue labels, and repo topics are traits (`<fork>`, `<bug>`, `<cli>`); an organization answers the `<org>` shorthand and the forge-neutral `<group>` alias. Direct addressing skips enumeration (`/rust-lang/rust/issues/12345`, closed included; listings show the API's default open set). Transport is `gh api` over subprocess (`QUARB_GH` overrides); lazy, cached, read-only. Target: `github:[OWNER[/REPO]]`.

`quarb-gitlab` GitLab, whose defining shape is the *deep tree*: groups nest arbitrarily and mix subgroups with projects as children, which the REST API walks by URL-encoded full paths and the arbor restores to a path — `/tesslab/instruments/gauge/issues/7`, one GET per segment. Project sections: `files` (content as value), `issues` / `mrs` (the open sets; direct addressing by iid reaches any state), `releases`, `pipelines` (jobs as children; statuses as traits — CI is half of what a GitLab project is). Code over social: no follows, stars as a count; the edge fabric is `member` (with access-level edge data — `$-::access`, `$-::role`), `parent` on forks (`<-parent` = the forks), and `author` / `assignee` / `reviewer`. Labels (scoped included), topics, visibility, and statuses are traits; a project answers the forge-neutral `<repo>` alias (`//*<repo>` sweeps either forge identically). Transport is `glab api` over subprocess (`QUARB_GLAB` overrides; hosts via glab config), so self-managed instances work as glab does. Read-only. Target: `gitlab:[PATH]`.

`quarb-git` The git repository driver: the object DAG as the arbor. `/branches`, `/tags`, `/HEAD` (refs alias their commits — same properties, same children), `/commits` (rev-syntax aliasing: `/commits/'HEAD~2'` navigates without enumeration; `/commits/*` batches one `rev-list --all`). A commit's children are its *tree* — time-travel file access, blob content as the node value. References are inherent: `::parent~>`, `->parent` (merges fan out), `<-parent` (children, from the enumeration). The diff surface: tree entries carry the `<changed>` trait when their path is in the commit's diff vs its first parent (trees by prefix), so `git log -- path` is `/commits/*[/path<changed>]`; commits answer `::changed` (the touched paths, deletions included) and `::::n-changed`. Plumbing over subprocess — no libgit2, zero new dependencies. Target: `git:PATH`. Read-only.

`quarb-atrep`	The atrep semantic-document adapter — the OQL homecoming. Mounts a deltos (`.atd`) or kanon (`.atk`) *through its dialektos*: sims named by their dialect names (`//section`, `//footnote`), genoses as traits, `::onym`/`::taxis`/ `::lemma` projections, deixes and onym-monosims as typed crosslinks with a precomputed reverse index (the first adapter to prebuild both `links` and `backlinks`), `<dangling>` as a broken-cross-reference linter. Mounting runs `kanonizo::tasso` (ordering only — onyms stay as authored; transclusion stays visible as edges). Depends on `quarb` and `atrep`.
`quarb-serve`	The *serve protocol* — the extension surface for tools the engine never links: a tool implements `AstAdapter` over its own model and hands it to `serve()` (three lines; non-Rust tools implement the protocol directly — nine operations over line-oriented JSON), and `qua serve:COMMAND` spawns it and mounts it beside every other source. LSP's shape: the protocol is the plugin system. Two wires, one protocol: the handshake is JSON (the universal floor) and advertises formats; every first-party server offers *kaiv* (`.daiv` frames, natively typed leaves under `/@serve`) and the client upgrades to it — the house format is the default wire, JSON the foreign-tool floor. Read-only by construction (no mutating operation exists). First external user: `cuj quarb-serve` (the cuj vault, mounted without qua knowing cuj). Reference server: `quarb-serve-json`; identity-tested against the direct adapter.

`quarb-sql`	The SQL bridge, both directions: `translate` imports a `SELECT` statement (WHERE, one JOIN via correlation and the witness, GROUP BY/HAVING, ORDER/LIMIT/DISTINCT, `LIKE` as case-insensitive regexes), and `export` walks a Quarb query's reflection arbor back to `SELECT` — witness joins become `JOIN ... ON`, grouped reductions become `GROUP BY`/`HAVING`; `~>` chains are refused by design (foreign keys live in the schema, not the query — spell `<=>` to export). Both directions differentially tested against SQLite; binaries `sql2quarb` / `quarb2sql`.
`qm`	The mailbox interface for the terminal — the first *library-path* client beside `qua`: it opens the Maildir adapter once per invocation, addresses messages as node handles, and phrases its subcommands (`ls`, `read`, `thread`, `from`, `grep`, `count`) as queries where a query is the natural shape, with `qm query` as the raw escape hatch. Actions are the tool's, not the engine's: `qm seen` performs the Maildir flag rename itself on the file handle the adapter exposes (`path_of`).

`qua`	The command-line tool. Picks the adapter from its argument (a directory is filesystem; `.csv`/`.tsv` is a table; a file or piped stdin is XML by extension or a leading `<?xml`, HTML by extension or a leading `<`, else JSON) and prints results. `--xpath` / `--jq` / `--sql` translate the query from XPath, jq, or SQL first; database inputs get *pushdown* — a query whose constructs are all in the verified-safe set (filtered selections, whole-table aggregates, witness joins) compiles to SQL and runs on the server, with rows ordered by the catalog key so output is identical to the scan path (`--no-pushdown` opts out; any refusal or error falls back silently, and `--explain` prints the decision — the pushed SQL, or the construct that kept the query on the scan path). When full pushdown refuses, *partial* pushdown pushes the leading row predicates as the table's fetch filter and runs the original query over the reduced set (gated: single reach, no crosslink/resolution axes, no metadata — anything that could observe the filtering). `--save FILE` materializes a result — `.db`/`.sqlite` as a table (`--as` names it), `.json` as a JSON array — both first-class inputs for later queries (the reference syntax is the existing file/mount mechanism); `--daiv` emits canonical kaiv (typed leaves with per-value provenance; needs the sibling `kaiv-rs` checkout).

Adapters are separate crates by design: `quarb-fs`, `quarb-json`, `quarb-html`, and `quarb-xml` share nothing but the `AstAdapter` trait, and a consumer pulls in only the adapters it needs. Each new adapter has confirmed the surface without changing it — the filesystem, a JSON tree, an HTML DOM, and an XML document are genuinely the same arbor to the engine.

2.2 The Execution Ladder

Database adapters evaluate on a three-rung ladder; every rung is *output-identical* to the one below it (differential tests hold this), so the rungs trade only latency and transfer:

1. **Full pushdown** (`quarb_sql::pushdown`, the exporter's strict mode): the whole query compiles to SQL when every construct is in the verified-safe set — WHERE-filtered selections, whole-table aggregates, witness joins. Strictness is the point: anything whose SQL semantics are not provably identical (LIKE case folding, truthiness, GROUP BY/DISTINCT order, collations, unordered LIMIT) refuses. Row order is part of the contract — the plan names the result table, and each driver appends `ORDER BY` its key from the catalog. Measured: COUNT over 500k PostgreSQL rows, 2.57 s scanned vs. 0.018 s pushed.

2. **Partial pushdown** (`quarb_sql::partial_pushdown`): the leading run of strictly-translatable row predicates becomes the table's fetch filter (each driver's lazy fetcher accepts a per-table WHERE), and the *original* query runs unchanged over the reduced set — the engine re-applies the pushed predicates, a no-op on pre-filtered rows, so there is no query rewriting. Gates: leading run only (a positional predicate before an expression one sees unfiltered rows), a single `/table/*` branch with no correlations, the table reached exactly once, no crosslink/resolution axes, no `:::/::::` metadata (each would observe the filtering). Since the JSON-column round (2026-07-31), refused predicates in the leading run degrade to *superset prefilters* instead of aborting the plan: fixed-path JSON comparisons emit per-dialect forms proven never to drop a matching row (SQLite typed extract with a `json_type='text'` escape hatch, PostgreSQL `jsonb_typeof` CASE with `::float8`; existence and `!=` floor at `json_type IS NOT NULL`) — extras are re-checked by the re-run, and a server-rejected prefilter surfaces through an eager prefetch and falls back to the scan rather than answering empty. Full JSON string-equality pushdown carries the dialect's validity guard (`json_valid` and kin) and refuses numeric-looking literals (Quarb's `value_eq` coerces those numerically; no extractor matches that). Measured: unpushable grouping over a 1%-selective filter, 2.31 s vs. 0.081 s; JSON numeric compare over 200k rows, 1.46 s scanned vs. 0.25 s prefiltered.

3. **Lazy scan**: catalog-eager, rows-lazy per table (Section on `quarb-relational`); correct for everything, always the fallback.

`qua --explain` prints the decision chain; `qua --no-pushdown` pins the scan rung; `qua --save FILE [--as NAME]` materializes any result as a first-class input (`.db` table / `.json` array) so expensive reductions run once and iteration proceeds locally. For metered engines (a future BigQuery adapter bills by bytes scanned), the ladder's transfer reductions are cost reductions.

2.3 Current Query Subset

The engine implements essentially the whole language. A query is a root-anchored sequence of hops, each an axis followed by a name:

Form	Meaning
`/name`, `//name`	**[live]** child / descendant named `name`
`\name`, `\\name`	**[live]** parent / ancestor named `name`
`>name`, `<name`	**[live]** next / previous sibling
`>>name`, `<<name`, `>>?`, `>>!`, `<<?`, `<<!`	**[live]** sibling reach: all following / preceding siblings at any distance, with proximal / distal quantifiers (matcher before reach, mirroring `//?`); covers CSS `~` and XPath following/preceding-sibling
`//?`, `//!`, `\\?`, `\\!`	**[live]** proximal (nearest) / distal (farthest) reach
`^`, `name$`	**[live]** root anchor / leaf anchor
`*`, `*.rs`, `~(re)`	**[live]** wildcard, glob, and regex name matching
`<code>`, `<a|b>`, `<a><b>`	**[live]** trait filters (alternation, conjunction)
`::`, `:::key`, `::::key`	**[live]** property, core-metadata, and adapter-metadata projection
`[::::size > 1K]`, `[/child]`, `[n]`, `[-1]`, `[2..5]`	**[live]** predicates: comparisons (`= != < <= > >=`), substring `*=`, regex `=~`/`!~`, structural conditions, `and`/`or`/`!`/`not`, positional index and range (negative counts from the end)
`::price * ::qty`, `7 div 2`, `idiv`, `mod`	**[live]** value expressions: arithmetic over numeric readings, in predicates, pipeline stages (`| ::a * ::b`), and pushes (`| .total(::a * ::b)` — a computed column); null propagation, exact integers, overflow to float
`//a || //b`, `| count`, `| sort | join(",")`	**[live]** union, and pipelines over a standard-library function set
`->target`, `<-target`, `->*`	**[live]** crosslink navigation (filesystem: symlinks)

`(/ul/li)+`, `(/p|/div)`, `(->mgr){1,3}!`, `/{2}`	**[live]** path patterns: groups, subpath alternation (strict and tolerated forms), quantifiers (`+ * {n} {m,n} {m,}`) with proximal/distal suffixes, the pattern dot wildcard, and the bare-operator sugar. Simple-path expansion (no node revisited within one path, the anchor included), capped by the adapter's quantifier bound (default 32; `--quantifier-bound` overrides)
`| upper`, `@| count`, `| .n`, `| $.n`	**[live]** per-capsa transform, whole-context aggregation, register push and recall
`@| sort_by(::age)`, `@| top(3, ::fare)`, `@| [..n]`, `| [::age > 30]`	**[live]** keyed aggregates (`sort_by`, `unique_by`, `min_by`/`max_by`, `top`/`bottom`; capsa-preserving, missing keys do not compete), whole-context positional selection, and per-capsa filters
`@| group(::k) | top(2, ::v) @| ungroup`, `$ord`	**[live]** value-keyed grouping (list topics + member capsae, keys as named regs), per-group keyed stages on the plain pipe, `ungroup` flattening, and the `$ordinal` operand (position in the current context, scope-threaded into filters)
`| %.`, `qua q.quarb`	**[live]** the register record view (named regs as a record: the row-wise table view, subsuming the planned `@.|`) and reflection: `.quarb` files open as the query's own arbor (`quarb::reflect`; vocabulary locked v1, enforced by the conformance corpus in `tests/vocabulary.rs`, version on the root as `::::vocabulary`)
`def &f($x): ...;`, `&f(::age)`, `--defs`, `--expand`	**[live]** fragments: parse-time expansion with call-by-name `$param` forms, defs files (`#` line comments stripped in `parse_defs`), and the unparser (`quarb::unparse`) rendering any parsed query back to canonical text (`--expand` = macroexpand); correlation chains and inline-pipe splices as bodies; `rec`/`group` field-name collisions are parse errors (`validate_record_convention`)

`| [::x =~ /(\w+)/] | %(k = $1), quai`	**[live]** match captures (filter-stage `=~` binds groups onto the capsa) and the interactive session (living-query lines with rollback, session defs, `:show`/`:undo`; retired from `qua` in favor of the standalone `quai` crate; `NAME=TARGET` mount aliases via `quarb_session::MountSpec` and in-session `:mount` via `Session::set_executor` — in-process mode only; the Python `Session`/kernel accrete defs-only cells through the `parse_defs` binding)
`@| window(-2..0) | mean, @| window(..0) | sum, @| shift(1, ::k)`	**[live]** window functions: offset-range spans over context neighbors as members (partial edges; `window(n)` = trailing count; open start = expanding), `shift` lag with own registers kept, and optional partition keys (per-group windows in original row order)
`"${::name} (${::age})", macro &m($p, @rest): body;`	**[live]** string interpolation (double-quoted `${expr}` holes, evaluated in the current scope; null splices empty) and procedural macros: the body runs at expansion time against the invocation's expansion arbor (argument forms as reflected subtrees, `::form` = each form's text), literal args bind by value, `@rest` collects the remainder, and the text results — joined, reparsed against strictly-earlier fragments — become the expansion (`--expand` = macroexpand)
`macro &pivot!($col): /data/row | $col @| unique | ...`	**[live]** data-aware macros: `!` on the definition mounts the queried dataset as `/data` in the expansion arbor, and every invocation spells the `!` (enforced both ways); outside a hole a parameter is the argument form (call-by-name), inside a hole its text — so a pivot reads distinct data values at expansion and generates a column per value; `--expand` with an input expands against it
`| .(//*.rs @| count), | .t(^/row @| count)`	**[live]** subcontexts (grouped aggregation); a `^`-anchored branch navigates from the arbor root, so a subcontext can consult the whole table (share-of-total columns)

`.c(^/row[::k = $$.k] @\| count),\| %($$1/name::; /amt::)`	**[live]** correlated subqueries: `$$` steps a capsa-scope operand out to the invoking capsa (`$$.name`, `$$_`, `$$ord`; the node form `$$::prop` / `$$/child::x` re-anchors on the invoking capsa's node; stacking `$$$` steps further) — wide crosstabs, groupwise transforms; and the correlation keeps its *witness*, so `$*k` in pipeline stages projects the joined side into output (`SELECT a.x, b.y`); the witness travels into every path a stage's operand or pipe hop navigates, so a nested step predicate (`(^/c/*[::b_id = $$1::id])`) reads the same thread's binding rather than re-searching the trace (2026-08-18)
`A <=> B[/k:: = $$/k:: and …]`	**[live]** correlation, *driver-first* (2026-07-25): the first expression drives (SQL's `FROM`); each joined expression carries its ON clause in its final step's brackets, reaching the driver as `$$…` and earlier entries as `$*k`; `<=>?` marks the expression it precedes nullable (`LEFT JOIN`); one tuple binds the whole predicate set, searched depth-first with per-entry pruning, and `$*N` may navigate from its context node (`$$1/field::`), so joins work where keys are children, e.g. JSON. Misplaced correlated references are parse errors, never silent nulls
`::ref~>`, `::'$ref'~>/city`	**[live]** cross-reference resolution (JSON: follow a `$ref` JSON Pointer)
`::ref<~`	**[live]** reverse resolution: every node whose `ref` resolves to this one (naive whole-arbor scan)

`$.r:::provenance`, `:::%provenance`, `:::@origin`, `--reproducible`	**[live]** value provenance (2026-09-06/07): every value carries its *origins* (node, stage; `Origins` bounded at eight, the rest counted) and the provenance entries of the nodes it was read from, kept in lockstep with record fields and list items (`Prov` in `exec.rs`, thread-local read frames). The value-side `:::` rung (`Operand::ValueMeta`) answers `provenance` (kaiv text), the shaped keys `%provenance` / `@provenance` (record / list: source, path, instant, instant-from, dpid), `origin` / `%origin` / `@origin` (node ids with the stage), `path`, `elided`. `resolved_provenance` fills the name-path within the source and completes the instant ladder node → source (mount target mtime) → access (invocation instant); `--reproducible` holds the two lower rungs back. `qua --kaiv` exports per field and per plural on the semicolon with `;+N` elision (`--kaiv-origins`)
`X:.r`, `X:@.`, `X:%.`, `X:(*.)`	**[live]** cross-capsa reads (2026-09-07): the register of the capsa *standing at* the node `X` reaches, read from the stage's input snapshot (`Operand::PeerReg`, a lazy `Peers` index by node); location, not lineage — after a fork nothing stands at the fork node, its children do; null outside a pipeline context. What lets one push read its neighbours: `<-link:%.` inside an expression body is the peer's whole register
`((!7))`, `((!$.v:::origin))`	**[live]** the node-id anchor (2026-09-07): `Anchor::Id` takes an expression; a number, a `%origin` record (by `id`), a list forks. `AstAdapter::has_node` guards it (default: probe parent/name under `catch_unwind`, since most adapters index a Vec by id). Ids must be deterministic across runs — minted from the node's identity in the source, never from walk order

`| .v(body){20}, {1;}, {m;n}`	**[live]** the quantified push (2026-09-07): `Stage::Repeat` wraps a push-family stage (push, expression push, subcontext, record and fields push); each round is one `apply_stage` over the whole context, so a round reads the previous round's snapshot; `{N}` exact, `{m;n}` / `{m;}` at least `m` then stop at a fixpoint (no node, topic, or `%.` view changed) or at `n` / the quantifier bound. PageRank over quarb.org is one such push; the stored column matches to 2×10^{-17}

The pipeline runs on *capsae*—the register system's unit, a node paired with a register (its breadcrumbs) and a current topic. A per-capsa `|` transforms each topic; `@|` reduces the whole context; `| .name` pushes the topic onto the register and `| $.name` recalls it; a subcontext `| .(expr)` evaluates `expr` from each node, reduces it to a scalar, and pushes it—grouped aggregation without leaving the query. A push quantified `{N}` repeats over the whole context, each round seeing the last (the iteration loop); `X:.r` reads the register of whichever capsa stands at the node `X` reaches, from the same snapshot; and every value remembers where it was read, so `:::` on a value answers its provenance the way `:::` on a node does.

Any other operator is lexed and rejected with a `not yet supported` error naming the offending construct, rather than silently mis-parsed. Everything in Section 3.3 falls into this category today.

Chapter 3

The Adapter Surface

An *adapter* maps a data source onto the Quarb arbor model — a tree backbone whose edges carry *names*, enriched with non-hierarchical crosslinks. The engine drives all navigation through the `AstAdapter` trait, so this trait *is* the contract between the language and the world.

3.1 The Live Trait

The methods the skeleton actually uses. Signatures are the real ones from `quarb/src/adapter.rs`:

```rust
/// An opaque handle to a node, minted and interpreted by the
/// adapter that produced it.
pub struct NodeId(pub u64);

pub trait AstAdapter {
    /// The root node -- the initial navigation context.
    fn root(&self) -> NodeId;

    /// The tree children of `node`, in document order.
    fn children(&self, node: NodeId) -> Vec<NodeId>;

    /// The name of `node` -- the label of its incoming tree edge.
    /// `None` when the adapter leaves a node unnamed (e.g. the
    /// filesystem root).
    fn name(&self, node: NodeId) -> Option<String>;

    /// The parent of `node`, or `None` for the root.
    fn parent(&self, _node: NodeId) -> Option<NodeId> { None }

    // --- projection surface (defaulted; implement what fits) ---

    /// Adapter-defined classifications, for `<trait>` filters.
    fn traits(&self, _node: NodeId) -> Vec<String> { Vec::new() }

    /// A named property `::prop`.
```

```
    fn property(&self, _node: NodeId, _name: &str) -> Option<Value> {
    None }

    /// The default projection, bare `::` (filesystem: file content).
    fn default_value(&self, _node: NodeId) -> Option<Value> { None }

    /// Adapter metadata `::::key` (filesystem: size, modified, …).
    fn metadata(&self, _node: NodeId, _key: &str) -> Option<Value> {
    None }

    /// Outgoing / incoming crosslinks as (label, node) pairs, for
    /// `->` / `<-` (filesystem: symlinks).
    fn links(&self, _node: NodeId) -> Vec<(String, NodeId)> { Vec::new
    () }
    fn backlinks(&self, _node: NodeId) -> Vec<(String, NodeId)> { Vec::
    new() }

    // --- the reference surface (defaulted) ---

    /// Resolve a cross-reference: `::prop-->hint` maps a property
    /// value to its in-document target (html: a `#fragment` href
    /// to the element with that id).
    fn resolve(&self, _n: NodeId, _prop: &str, _hint: Option<&str>)
        -> Option<NodeId> { None }

    /// The external reference a property holds -- an
    /// adapter-defined identifier of a document outside the arbor
    /// (html: the absolute URL, relative hrefs joined against the
    /// document's declared URL). On an in-document miss the engine
    /// consults the mount layer's reference targets (identifier ->
    /// mounted root) and lands, or records the identifier among
    /// the run's unresolved external references for the host's
    /// acquisition loop. The engine performs no IO, ever.
    fn external_ref(&self, _n: NodeId, _prop: &str, _hint: Option<&str
    >)
        -> Option<String> { None }

    /// The element `fragment` names inside `node`'s document --
    /// the landing rung of a fragment-carrying external reference.
    fn resolve_fragment(&self, _n: NodeId, _frag: &str)
        -> Option<NodeId> { None }

    /// The property carrying the node's own reference, for the
    /// bare arrow `//a-->` (html: an anchor's href); and the
    /// relation an edge carries (html: `rel`), the label falling
    /// back to the property name.
    fn ref_property(&self, _n: NodeId) -> Option<String> { None }
    fn ref_label(&self, _n: NodeId, _prop: &str) -> Option<String> {
    None }

    // --- the index hooks (defaulted; an indexed store answers) ---

    /// The document an external identifier names (a URL -> its
    /// page), the arrow's landing rung after the mount layer.
    fn document_by_ref(&self, _id: &str) -> Option<NodeId> { None }
    /// `<--` from an index instead of a whole-arbor scan.
    fn reverse_resolve(&self, _n: NodeId, _prop: Option<&str>,
```

```
        _hint: Option<&str>) -> Option<Vec<NodeId>> { None }
    /// `//name` below `node` from an index, with each hit's depth;
    /// `None` means walk. The answer must equal the walk's.
    fn descendants_named(&self, _n: NodeId, _name: &str)
        -> Option<Vec<(NodeId, usize)>> { None }
    /// A batching hint before a link sweep over `nodes`.
    fn prefetch_links(&self, _nodes: &[NodeId], _dir: LinkDir) {}

    // --- identity and provenance (defaulted) ---

    /// Whether `node` is a node of this arbor -- the guard of the
    /// node-id anchor `((!expr))`. Default: probe parent and name
    /// under catch_unwind.
    fn has_node(&self, node: NodeId) -> bool { ... }
    /// What the source recorded about `node`: source, path within
    /// it, instant (and which rung it came from), dpid; a list when
    /// several sources stand behind one node (mounts, grafts,
    /// models layer entry by entry).
    fn provenance(&self, _n: NodeId) -> Provenance { ... }
    fn provenance_list(&self, n: NodeId) -> ProvenanceList { ... }
}
```

The navigation methods are required; the projection methods carry defaults, so an adapter implements only what its domain offers. Core metadata (`:::key` — index, depth, counts, `is-leaf`, ...) is computed by the engine from the navigation surface, so no adapter implements it.

Three observations that the filesystem work has already forced:

- **Names live on edges, not nodes.** `name` returns the label of a node's *incoming* edge — its role in its parent — exactly as the specification's edge-name model requires. For the filesystem this is simply the filename.
- **NodeId is opaque and adapter-private.** The engine never inspects the u64; it only passes ids back to the adapter. Each adapter chooses its own identity scheme (the filesystem adapter uses an interning arena index).
- **New trait methods need their forwarders.** The run path wraps every adapter in decorators (`WithNow`, `AllowShell`), the session in `Dyn`, mounts in `MountAdapter`/`Shared`, models in `ModelAdapter` – a defaulted method silently answers `None` through any wrapper that does not forward it. When adding to the trait, sweep every delegating impl. The case in point (2026-09-07): `Dyn` had never forwarded `answers_to`, so a web level behind the session's boxed door answered `//page` with nothing – its rows answer to their kind through that method. It forwards the whole surface now.
- **The root may be unnamed.** `name` returns `Option` precisely so an adapter can leave the root without a name, as the filesystem root `/` does.

3.2 Worked Example: the Filesystem Adapter

`quarb-fs` implements the live trait over a directory tree. It materializes the tree once, at construction, into arrays indexed by `NodeId` (path, children, parent); the trait methods are then trivial lookups:

```
impl AstAdapter for FsAdapter {
    fn root(&self) -> NodeId { self.root }
    fn children(&self, node: NodeId) -> Vec<NodeId> { self.children[
    node].clone() }
    fn name(&self, node: NodeId) -> Option<String> {
        if node == self.root { return None; }   // "/" is unnamed
        self.paths[node].file_name().map(|n| n.to_string_lossy().into()
    )
    }
    fn parent(&self, node: NodeId) -> Option<NodeId> { self.parents[
    node] }
}
```

Hidden entries and directory links [live]. An entry whose name starts with a dot carries the `<hidden>` trait (the mount root never does); such entries are in the tree only under `--hidden`. A symlink is a node with a `target` crosslink, and a symlink to a directory of the tree is also transparent to an explicit child step, as it is in the file system: with `latest -> drafts`, `/latest/a.md` is the node `/drafts/a.md`, the same one `/latest->*/a.md` reaches. The trait method behind this is `child_host(node)`, the node whose children a child step reads; it defaults to the node itself, and the wrapping adapters (mounts, grafts, models, reflection) forward it. The descendant walk reads `children` alone, so `//` sees each file once and a cycle of links cannot trap it. A broken link, and a link whose target lies outside the mounted tree, stay leaves.

Ignore rules [live]. The materializing walk uses `ignore::WalkBuilder`—the same machinery ripgrep uses—so by default the adapter respects `.gitignore`/`.ignore` files and skips hidden entries. This came directly from running the skeleton: `qua '//*.rs'` in the engine's own tree first returned a build artifact under `target/`, because nothing yet excluded it. The adapter now honors ignore rules by default, and `qua` exposes `--hidden` and `--no-ignore` to opt back in. The walk is currently eager (a targeted query still walks the whole tree); a lazy traversal with cascading ignore rules is a planned optimization. This is the pattern the guide follows: a concrete problem, surfaced by running the code, drives the next slice.

3.3 The Planned Surface

The rest of the arbor model, specified in the language but not yet built. Each becomes a live trait method (or a related capability) as its engine slice

lands. The intended shapes:

Capability	Query	Adapter surface (sketch)
Pattern search	`=>`	`search(pattern) -> Vec<NodeId>`.

These signatures are provisional. They are recorded here so the direction is legible, not to freeze a contract — the filesystem adapter (and the JSON adapter after it) will reshape them as they are implemented.

Chapter 4

Building and Running

```
# from the engine workspace root
cargo build            # build the whole workspace
cargo test             # unit + integration + doc tests

# query a directory tree
cargo run -p qua -- '//*.rs' src/     # all Rust files under src/
cargo run -p qua -- '/README.md'      # a single top-level file

# query a document
cargo run -p qua -- '/users/0/name::' data.json
cargo run -p qua -- '//book[::pages > 200]::::id' catalog.xml
cargo run -p qua -- '//a::href' page.html
```

`qua <QUERY> [PATH]` runs `QUERY` against the directory `PATH` (default: the current directory) and prints each matching path on its own line, so it composes in shell pipelines alongside `sort`, `uniq`, and friends.

Chapter 5

Planned Integration Paths

Beyond the direct `AstAdapter` implementation shown above, these integration paths are described in the specification but **[planned]** (none is implemented yet):

- **Petgraph adapter.** A ready-made adapter over a `petgraph::Graph`, for Rust callers who transform their AST into a graph once and query it at speed. See `petgraph-adapter.md` in this project for the design note.
- **Python bindings.** A `quarb` Python package exposing the engine and an `AstAdapter` base class.
- **C FFI.** A C ABI for embedding the engine in other languages.

Each will be added once the core language surface is substantial enough to be worth binding.